Does a Robot's Failure Monitor Still Work After a Speed-Up?

Anonymous Author(s)
Affiliation
Address
email

Abstract: Robots controlled by vision-language-action (VLA) models are often paired with a failure monitor: a lightweight check that raises an alarm when a task seems to be going wrong. The monitor's alarm threshold is tuned on the model as originally trained, but the deployed model is often a sped-up version, so we ask whether the monitor still works after that swap. We tune four common monitors on the open SmolVLA model, then run sped-up versions of it on the LIBERO-Long benchmark with the monitors left unchanged. Building each action plan in 2 refinement steps instead of 10 makes the model 2.5× faster with no loss of success (27% before, 32% after). Yet it disables a monitor that checks whether the robot's successive plans agree: that monitor caught about half of all failures before and catches none after, because the faster model's plans agree more closely, which pulls every score below the old threshold. Counting false alarms, the only thing the threshold's tuning guarantees, does not flag this, because false alarms drop to zero too. Rounding the model's weights to fewer bits nudges a different monitor, one that reads the model's visual and language features, toward reporting "safe". After any speed-up, a monitor should be re-tested; comparing its scores on successful runs before and after the speed-up already exposes the shift, without a single failure.

Keywords: VLA, acceleration, quantization, runtime monitoring, conformal prediction

1 Introduction

A *failure monitor* watches a robot while it works and raises an alarm when the current attempt looks likely to fail, so that a person or a fallback controller can step in. For learned robot policies, including VLAs, a monitor is usually a small scoring function that reads either the policy's internal state or the actions it plans; an alarm fires when the score crosses a threshold [1, 2, 3, 4]. A common recipe sets the threshold using only successful runs of the policy, so that at most a fixed fraction of them, say 10%, would raise an alarm [1, 3, 4]. This conformal guarantee [5] limits false alarms, but only while the deployed policy behaves like the one the threshold was tuned on, and it says nothing about how many failures get caught.

The deployed policy is often not the policy as trained. To run fast enough on a robot it is sped up: its weights are stored with fewer bits (quantization), it is pruned, or it takes fewer steps to produce each action [6, 7, 8]. Papers on these speed-ups check task success; none of them evaluates the monitor. A monitor can break in two ways: its scores can stop ranking failures above successes, or they can keep ranking well while drifting past the fixed threshold. For language-model monitors, fine-tuning often degrades ranking (recoverably, by realignment) while quantization largely preserves it [9]. Concurrent work finds the second kind of failure for a VLA probe under 4-bit weights, by replaying the original policy's recorded observations through the compressed model [10].

Submitted to the CoRL 2026 Workshop on Efficient Foundation Models for Real-Time Embodied AI. Do not distribute.

We test the deployed setup directly: each sped-up policy runs on the task itself, so its own actions decide what it sees next, while four unchanged monitors watch it.

2 Setup

Robot policy and task. SmolVLA [11] is an open VLA with ∼450M parameters. A vision-language *backbone* (SmolVLM2) reads the camera images and the instruction. An *action expert* then plans the next 50 actions by flow matching: like a diffusion model, it turns random noise into a plan over 10 refinement steps. The robot executes the first action and replans at the next timestep. We use the public LIBERO checkpoint with its own settings, run through LeRobot, on LIBERO-Long [12], a simulated suite of 10 long-horizon manipulation tasks. Each episode starts from one of 50 fixed initial states per task and fails if the task is not done within 520 timesteps. The original policy succeeds in 32% of all 500 episodes (27% on the test set), below the 71% the SmolVLA paper reports; a public reproduction that executes 10 actions per plan reports 56%.[1] We could not explain this gap. On 10 LIBERO-Spatial episodes, our rollout code and LeRobot's own evaluator both succeed 6 times.

Speed-ups. **2 steps**: 2 refinement steps instead of 10, weights untouched; the median time per policy call (batch of 10, one L4 GPU) falls from 0.81 s to 0.32 s. **Weight quantization** rounds the weights of the 521 linear layers in the backbone and action expert to 8 or 4 bits; weights in a group share one scale, and bigger groups round more coarsely. **int8** uses one scale per output channel, **int4-g32** one per 32 weights, and **int4-coarse** one per 128 weights where the layer width allows and one per row otherwise. The rounding is simulated, so it reproduces low-bit accuracy but not low-bit speed.

Monitors. We fit four monitors once, on runs of the original policy, and never change them. Two read the backbone and two read the action expert.

- **VLM probe**: a linear classifier on the backbone's last-layer features that predicts whether the episode will fail.
- **Mahalanobis**: how far the backbone's features lie from those of successful training runs; it needs no failure examples (a backbone variant of VLA-FAIL's detector [2]).
- **Expert probe**: the same classifier on the action expert's internal state, which is where SAFE reads flow-matching VLAs [1]; a linear stand-in for SAFE.
- **Action consistency**: how much each new 50-action plan disagrees with the previous one where they overlap, after VLA-FAIL [2]. Sentinel [4] instead compares distributions of many sampled plans.

A monitor's score at timestep t is the average of its per-timestep scores so far, and the alarm fires the first time that average exceeds the threshold after a 10-timestep warm-up. The threshold is set on successful runs of the original policy so that at most 10% of them would raise an alarm.

Protocol. Initial states 0–29 of every task train the monitors, 30–39 set the thresholds, and 40–49 form the test set: 100 episodes per variant, all with the same initial states and random seed. The hypotheses, outcomes (including re-tuning), splits and monitors were pre-registered before the main data collection. The int4-g32 variant and all other analyses (paired tests, best possible thresholds, score shifts, checks) were added afterwards and are exploratory.

3 Results

Two refinement steps disable the action-consistency monitor. The 2-step policy succeeds in 32% of test episodes, against 27% for the original.

[1] https://github.com/huggingface/lerobot/issues/3264

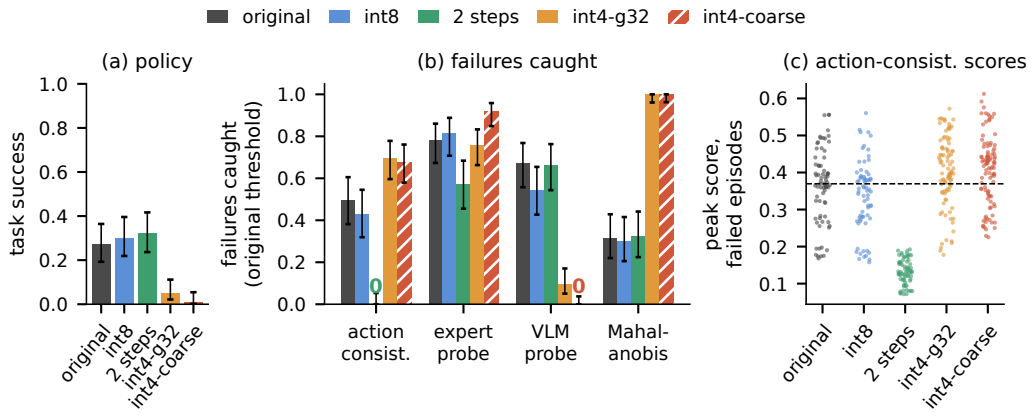


Figure 1: **(a)** Task success. **(b)** Fraction of failures each monitor catches with the original threshold. Both int4 variants break the policy, so their high bars do not mean good monitoring. **(c)** Each dot is a failed episode's peak action-consistency score; dots below the dashed original threshold are missed. Error bars are Wilson 95% intervals over the 100 test episodes (a) or the failed ones (b), not clustered by task.

Table 1: Original → 2-step policy, test set. *Caught*, *false alarms*: with the original threshold. *Best possible*: most failures any threshold catches at ≤10% false alarms, chosen with hindsight (caught can exceed it when the original threshold gives over 10% false alarms). *AUROC*: chance a random failure outscores a random success. *Re-tuned*: threshold reset on the 2-step policy's successful runs.

monitor	failures caught	false alarms	best possible	AUROC	re-tuned: caught / false al.
action consistency	.49→**.00**	.15→.00	.44→.34	.77→.72	.26 / .03
expert probe	.78→.57	.11→.00	.77→.71	.95→.94	.66 / .06
VLM probe	.67→.66	.04→.00	.81→.87	.95→.96	.60 / .00
Mahalanobis	.32→.32	.07→.06	.36→.38	.85→.86	.29 / .06

- **Action consistency** caught 49% of the original's failures (36 of 73) and catches none of the 2-step policy's 68 (Fig. 1b,c). On initial states where both fail, it misses 30 failures it used to catch and gains none (p=2×10^{-9}, paired McNemar; 95% interval over tasks and thresholds for the change: -0.92 to -0.25). It stays at zero with a stricter 5% false-alarm target.
- **The expert probe** drops from 78% to 57% (14 lost, 0 gained), and its median alarm moves from timestep 10, the earliest allowed, to 124. But the interval over tasks (-0.41 to $+0.08$) includes zero and the drop vanishes at a 5% target, so we treat it as suggestive.
- **The backbone monitors** barely move: the VLM probe catches 66% against 67%, and Mahalanobis 32% against 32%. Fewer refinement steps change only the action expert; on the first timestep, where both versions see the same image, the backbone's features are bit-for-bit identical.[2]

The scores still rank failures; the threshold is what breaks.

- **The score shrinks by a roughly constant factor.** With 2 steps, successive plans agree more. On the same initial state, action disagreement over the first 30 timesteps is 0.42 times its original value (middle half of episodes: 0.40–0.44), and 0.43 for runs that succeed in both versions.

[2]On 6 of the 10 tasks, and on 19 of 20 episodes of a seventh. On the other 3, the original's first-timestep features differ slightly, likely from GPU kernel autotuning. Restricted to the 6 matched tasks, the detection results hold except int8's.

- **Ranking largely survives** (Table 1): AUROC goes from 0.77 to 0.72, and with hindsight a threshold could still catch 34% of failures. But the old threshold now sits above every score, and re-tuning it on the 2-step policy's successes recovers only 26%, against 49% originally.
- **The failure looks like an improvement.** False alarms fall to zero (from 15% for action consistency, 11% for the expert probe). A team that checks the false-alarm rate, the one thing the threshold guarantees, would see the monitor getting better.

Concurrent work reports the same pattern, ranking intact but threshold wrong, for quantized probes whose threshold is set on failures [10]. With a threshold set on successes, as is common, the shift shows up as fewer false alarms and hides the loss. Here the pattern also hits a monitor with no trained weights.

Weight quantization nudges the VLM probe toward "safe". On the first timestep, where every variant sees the same image, the VLM probe's score moves toward "safe" by 0.29 standard deviations of the original's first-timestep scores under int8 (in all 120 matched episodes), 2.3 under int4-g32 and 4.6 under int4-coarse. The share of failures it catches falls from 67% to 54% (int8; 13 lost, 3 gained, p=0.02, but not significant on the matched tasks), 9% and 0%. Under int8, task success and false alarms show no detectable change. Both int4 settings break the policy itself (5% and 1% success): under int4-coarse the probe stays silent through all 99 failures, while Mahalanobis alarms on every int4 episode, successes included, so it detects the compression rather than the failures.

Checks. *Episode length:* failed episodes run to the time limit, which gives a monitor more chances to alarm [10]. Counting only alarms before timestep 200 lowers the original detection rates (VLM probe 0.67→0.48, action consistency 0.49→0.27), but both zeros stay at zero. *Task identity:* two tasks never succeed, so a probe could score well by recognizing them; within single tasks the original AUROC is still 0.96 (VLM probe) and 0.98 (expert probe), and dropping those tasks changes no conclusion. *Pre-registration:* both primary hypotheses concerned int4 and were uninformative, as int4-coarse left one successful test episode. Of the secondary ones, 2 steps moved the expert probe's false-alarm rate more than int8 did (held), and int8 moved every false-alarm rate by at most 0.05 but one AUROC by 0.08 (failed). We did not predict the detection collapse.

4 Related work and discussion

Related work. Monitors for VLAs and other learned policies are evaluated with the policy held fixed [1, 2, 3, 4]; even then, a SAFE-style monitor on held-out tasks can far exceed its false-alarm target [13]. SAFECAST [14] studies thresholds under changes to the robot's inputs. Monitor staleness under model updates has been studied for language models [9] and, concurrently, for a quantized VLA replayed on recorded inputs, repaired by realigning its activations [10]. Speed-up studies measure success, speed and motion quality [6, 7, 15]. CARE could not certify fewer flow steps for $\pi_{0.5}$ on LIBERO-Long [8]; for SmolVLA we saw no loss of success, only of monitoring.

Takeaways. Treat every speed-up as invalidating a monitor's threshold, even when task success holds. Compare the monitor's scores on successful runs before and after: here the shrink is plain (median 0.27 vs. 0.11) without a single failure. Then re-test detection on real failures of the sped-up policy. Expect a monitor to break where its input changes.

Limitations. One policy, one task suite and one seed, with 100 test episodes per variant. The original policy's success rate is low, and quantization is simulated. Our monitors are simplified: SAFE, for example, uses a time-varying threshold. We did not test why 2-step plans agree more; plausibly, fewer refinement steps give less varied plans. int4-coarse was an unintended setting that code review caught.

References

[1] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, et al. SAFE: Multitask failure detection for vision-language-action models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[2] F. Seligmann, E. Gospodinov, E. U. Dincer, and G. Neumann. VLA-FAIL: Efficient task failure detection for finetuned vision-language-action models. In *Conference on Robot Learning*, 2026.

[3] R. Römer, A. Kobras, L. Worbis, and A. P. Schoellig. Failure prediction at runtime for generative robot policies. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[4] C. Agia, R. Sinha, J. Yang, Z.-a. Cao, R. Antonova, M. Pavone, and J. Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Conference on Robot Learning*, 2024.

[5] A. N. Angelopoulos and S. Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[6] J. Xu, Q. Jin, M. Chen, S. Wang, Y. Sui, and Y. Shi. VLAQuantBench: Closed-loop evaluation of post-training quantization for vision-language-action models. *arXiv preprint arXiv:2609.25376*, 2026.

[7] J. Zhang, Y. Hsieh, Z. Wan, H. Lin, X. Wang, Z. Wang, et al. QuantVLA: Scale-calibrated post-training quantization for vision-language-action models. *arXiv preprint arXiv:2602.20309*, 2026.

[8] R. Liu, T. Zheng, J. Gu, and Z. Wang. CARE: Certifying acceleration for vision-language-action inference. *arXiv preprint arXiv:2610.08917*, 2026.

[9] E. Duan. Do activation monitors survive model updates? Benchmarking, predicting, and repairing activation-monitor staleness. *arXiv preprint arXiv:2606.15980*, 2026.

[10] Anonymous. Rank-preserving, threshold-breaking: Post-training silently decalibrates inherited safety monitors in VLA policies. NeurIPS 2026 workshop submission, OpenReview `atOkXTzFCL`, 2026.

[11] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

[12] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks Track*, 2023.

[13] D. Ahmedt-Aristizabal, A. Khamis, M. A. Armin, and L. Petersson. RESCUE: Is this alarm worth acting on? Counterfactual evaluation of VLA runtime monitors. *arXiv preprint arXiv:2610.13516*, 2026.

[14] H. Rajaprakash, A. Prajapati, R. Xue, A. Anwar, and J. Thomason. SAFECAST: Robust failure detection for VLA policies with contrast-set training and calibration. *arXiv preprint arXiv:2608.04246*, 2026.

[15] Z. Li, H. Yang, Z. Chen, Y. Chen, et al. From inference efficiency to embodied efficiency: Revisiting efficiency metrics for vision-language-action models. *arXiv preprint arXiv:2603.19131*, 2026.